
HyperQuote

Documentation System Plan

Complete technical documentation plan for the HyperQuote RFQ
liquidity coordination protocol on HyperEVM.

Contents

1. [Documentation Sitemap](#)
2. [Page Descriptions](#)
3. [Required Diagrams](#)
4. [API Documentation Plan](#)
5. [Smart Contract Documentation Plan](#)
6. [Risk Documentation Plan](#)

HyperQuote Documentation System

Complete technical documentation plan for the HyperQuote RFQ liquidity coordination protocol on HyperEVM.

1. Documentation Sitemap

```
docs/  
  introduction/  
    what-is-hyperquote.md  
    how-rfq-works.md  
    architecture-overview.md  
    supported-tokens.md  
    glossary.md  
  
  getting-started/  
    connect-wallet.md  
    your-first-trade.md  
    become-a-maker.md  
    environment-setup.md  
  
  trading/  
    requesting-a-quote.md  
    quote-comparison.md  
    executing-a-fill.md  
    exact-in-vs-exact-out.md  
    public-vs-private-rfqs.md  
    rfq-lifecycle.md  
    venue-comparison-engine.md  
    price-improvement.md  
    supported-pairs.md  
  
  options/  
    options-overview.md  
    covered-calls.md  
    cash-secured-puts.md  
    strike-and-expiry-rules.md  
    collateral-requirements.md  
    options-rfq-flow.md  
    settlement-and-expiry.md  
    position-management.md  
    settlement-oracle.md  
  
  makers/  
    maker-overview.md  
    relay-connection.md  
    receiving-rfqs.md  
    submitting-quotes.md  
    eip712-signing.md  
    nonce-management.md  
    pricing-strategies.md  
    risk-management.md  
    maker-sdk-quickstart.md  
    maker-sdk-reference.md  
    auto-quoting-bot.md  
    private-rfq-routing.md  
    maker-reliability-score.md
```

- venue-comparison/
 - how-venue-comparison-works.md
 - hypercore-spot.md
 - hyperevm-dex.md
 - multi-hop-routing.md
 - partial-fills.md
 - benchmark-methodology.md

- smart-contracts/
 - contract-architecture.md
 - spot-rfq-contract.md
 - options-engine.md
 - settlement-publisher.md
 - quote-library.md
 - collateral-math.md
 - contract-addresses.md
 - security-model.md

- api/
 - api-overview.md
 - authentication.md
 - agent-registration.md
 - rfq-endpoints.md
 - quote-endpoints.md
 - fill-endpoints.md
 - feed-and-stream.md
 - leaderboard-endpoints.md
 - league-endpoints.md
 - badge-endpoints.md
 - profile-endpoints.md
 - benchmark-endpoints.md
 - sor-endpoints.md
 - relay-websocket-protocol.md
 - terminal-api.md
 - rate-limits.md
 - error-codes.md

- fees/
 - fee-structure.md
 - maker-fees.md
 - taker-fees.md
 - keeper-fees-options.md

- points/
 - points-overview.md
 - base-points-formula.md
 - multipliers.md
 - nft-boosts.md
 - anti-gaming-guards.md
 - points-faq.md

- league/
 - league-overview.md
 - maker-scoring.md
 - taker-scoring.md
 - ranking-periods.md
 - league-activity-feed.md

- risk/
 - risk-overview.md
 - smart-contract-risk.md
 - liquidity-risk.md
 - quote-expiry-risk.md
 - oracle-risk.md
 - relay-dependency.md
 - volatility-risk.md
 - routing-limitations.md
 - counterparty-risk.md

- faq/

```
general-faq.md
taker-faq.md
maker-faq.md
options-faq.md
technical-faq.md
```

```
links/
  official-links.md
  contract-addresses.md
  brand-assets.md
```

2. Page Descriptions

Introduction

what-is-hyperquote.md

- **Purpose:** Define HyperQuote and its position in the HyperEVM ecosystem.
- **Covers:** Protocol summary, RFQ model vs AMM/CLOB, value proposition for takers (price improvement on size) and makers (selective quoting without passive LP risk), supported products (spot + options), atomic on-chain settlement.
- **Audience:** All users.

how-rfq-works.md

- **Purpose:** Explain the Request-for-Quote execution model at a conceptual level.
- **Covers:** What an RFQ is, how it differs from limit orders and AMM swaps, the request-compete-fill lifecycle, role of the relay, role of on-chain settlement, why RFQ is suited to large trades.
- **Audience:** All users.

architecture-overview.md

- **Purpose:** Describe the technical architecture of the protocol.
- **Covers:** System diagram (UI, relay, smart contracts, venues, terminal), data flow from RFQ creation to on-chain fill, WebSocket relay role, venue comparison engine, off-chain vs on-chain components, chain ID and RPC configuration.
- **Audience:** Developers, makers.

supported-tokens.md

- **Purpose:** List all tokens available for RFQ trading.
- **Covers:** Core tokens (HYPE, USDC, kHYPE, USD■0, PURR, USDH, etc.), token tiers (core, verified, unverified), verification sources (manual, spotMeta, PRJX), native HYPE vs wHYPE settlement wrapping, token address registry.
- **Audience:** All users.

glossary.md

- **Purpose:** Define protocol-specific terminology.
 - **Covers:** RFQ, maker, taker, fill, quote, nonce, EIP-712, settlement, expiry, venue, slippage, price improvement, notional, pips, basis points, keeper, oracle.
 - **Audience:** All users.
-

Getting Started

connect-wallet.md

- **Purpose:** Walk through wallet connection and chain setup.
- **Covers:** Supported wallets (MetaMask, WalletConnect, injected), adding HyperEVM network (chain ID, RPC URL, block explorer), native HYPE currency, token approval flow.
- **Audience:** Takers, makers.

your-first-trade.md

- **Purpose:** Step-by-step guide for a taker's first RFQ trade.
- **Covers:** Selecting tokens, entering amount, choosing exact-in vs exact-out, submitting RFQ, waiting for quotes, comparing quotes against venues, executing fill, confirming on-chain settlement.
- **Audience:** Takers.

become-a-maker.md

- **Purpose:** Step-by-step guide for setting up as a maker.
- **Covers:** Connecting to the maker dashboard, relay WebSocket connection, receiving RFQ broadcasts, building a quote, signing with EIP-712, submitting via relay, monitoring fill status.
- **Audience:** Makers.

environment-setup.md

- **Purpose:** Developer environment setup for running the protocol locally.
 - **Covers:** Monorepo structure (apps/hyperquote-ui, packages/sdk-maker, services/relay, services/terminal-api), npm workspace commands, .env.local configuration, preflight checks, Foundry setup for contracts, running dev server + relay + terminal.
 - **Audience:** Developers.
-

Trading (Takers)

requesting-a-quote.md

- **Purpose:** Explain how to create and submit an RFQ.

-
- **Covers:** Token pair selection, amount input, exact-in vs exact-out, visibility selection (public/private/intra-taker), TTL (time-to-live), RFQ submission via contract, rfqId generation (deterministic keccak256), EIP-191 signature requirement, rate limits (10 public / 20 private per wallet).
 - **Audience:** Takers.

quote-comparison.md

- **Purpose:** Explain how incoming maker quotes are displayed and compared.
- **Covers:** Quote card anatomy (maker address, amountOut, expiry, signature status), validation states (valid, expiring, expired, invalid_sig), sorting by best price, fee deduction display (feePips), comparison against venue benchmarks.
- **Audience:** Takers.

executing-a-fill.md

- **Purpose:** Walk through the fill execution process.
- **Covers:** Selecting best quote, token approval step (ERC-20 approve), fill transaction submission, atomic on-chain settlement, transaction confirmation, execution panel states (approving, filling, confirmed, error).
- **Audience:** Takers.

exact-in-vs-exact-out.md

- **Purpose:** Explain the two quoting modes.
- **Covers:** Exact-in (specify input amount, receive variable output), exact-out (specify desired output, pay variable input), when to use each, how makers price each mode, impact on venue comparison benchmarks.
- **Audience:** Takers.

public-vs-private-rfqs.md

- **Purpose:** Explain RFQ visibility options.
- **Covers:** Public RFQs (broadcast to all makers), private RFQs (selective maker routing via allowedMakers list), intra-taker mode (self-quote), privacy benefits for large trades (no market signal), points multiplier for private fills (1.10x on fills >= \$50k).
- **Audience:** Takers, makers.

rfq-lifecycle.md

- **Purpose:** Document the complete lifecycle of an RFQ from creation to settlement.
- **Covers:** State machine (created, open, quoted, filling, filled, expired, cancelled), TTL countdown, quote accumulation window, fill execution, cancellation rules, on-chain finality.
- **Audience:** All users.

venue-comparison-engine.md

- **Purpose:** Explain how RFQ quotes are benchmarked against other venues.

- **Covers:** Three venues (HyperCore Spot, HyperEVM DEX, RFQ), comparison methodology, price improvement calculation, slippage estimation, how venue results are displayed (success/partial/failure), refresh interval.
- **Audience:** Takers.

price-improvement.md

- **Purpose:** Define and explain price improvement metrics.
- **Covers:** Price improvement formula (RFQ output vs best venue output, denominator = venue amount), improvement basis points, display in quote cards and savings banner, how improvement feeds into points calculation.
- **Audience:** Takers.

supported-pairs.md

- **Purpose:** Document which token pairs are available.
 - **Covers:** Core pairs, stable pairs, token tier requirements (core and verified tokens), native HYPE wrapping to wHYPE for settlement, pair-specific venue availability.
 - **Audience:** Takers.
-

Options

options-overview.md

- **Purpose:** Introduce HyperQuote Options product.
- **Covers:** Supported instruments (covered calls, cash-secured puts), underlying tokens (wHYPE in V1), collateral tokens (USDC, USDH, USD■0), expiry constraints (24h min, 90 days max, 08:00 UTC snap), ERC-721 position NFTs, physical settlement model.
- **Audience:** All users.

covered-calls.md

- **Purpose:** Explain covered call mechanics.
- **Covers:** What a covered call is, seller locks underlying (wHYPE), buyer pays premium, ITM settlement (buyer pays strike in stablecoin, receives underlying), OTM expiry (seller gets underlying back), collateral calculation (quantity of underlying).
- **Audience:** Takers, makers.

cash-secured-puts.md

- **Purpose:** Explain cash-secured put mechanics.
- **Covers:** What a CSP is, seller locks stablecoin collateral, buyer pays premium, ITM settlement (buyer delivers underlying, receives stablecoin), OTM expiry (seller gets stablecoin back), collateral formula (strike x quantity / $10^{(18 + \text{underlyingDecimals} - \text{collateralDecimals})}$).
- **Audience:** Takers, makers.

strike-and-expiry-rules.md

- **Purpose:** Document constraints on strike prices and expiry dates.
- **Covers:** Strike price format (1e18 fixed-point), expiry must be at 08:00 UTC (28800 seconds into day), minimum 24 hours from now, maximum 90 days, how the UI snaps expiry to valid timestamps.
- **Audience:** Takers, makers, developers.

collateral-requirements.md

- **Purpose:** Detail collateral locking and math.
- **Covers:** CollateralMath library functions (putCollateralRequired, callCollateralRequired, callSettlementCost, notional), ceiling division for precision, decimal scaling between underlying (18 dec) and collateral (6 dec), approval amounts.
- **Audience:** Takers, makers, developers.

options-rfq-flow.md

- **Purpose:** Document the options RFQ request-to-execution flow.
- **Covers:** Taker submits RFQ (specifying isCall, strike, quantity, expiry, minPremium), EIP-191 signature of rfqId, relay broadcasts to makers, makers respond with signed EIP-712 quotes, taker calls OptionsEngine.execute(quote, makerSig), contract verifies signature + nonce + expiry + allowlists, mints ERC-721 position to buyer, transfers premium seller-to-buyer, locks collateral.
- **Audience:** All users.

settlement-and-expiry.md

- **Purpose:** Explain post-execution settlement and expiry flows.
- **Covers:** Settlement window (24 hours after expiry), ITM settlement (keeper calls settle, oracle provides settlement price, physical delivery occurs), OTM expiry (anyone calls expirePosition after settlement window, collateral returned to seller), keeper fee (keeperBps of notional, max cap), position state transitions (Active -> Settled or Expired).
- **Audience:** All users.

position-management.md

- **Purpose:** Explain the positions tracking interface.
- **Covers:** Position lifecycle states (active, pending_expiry, expired, settled), positions table with expandable detail, summary strip (total notional, count, PnL), expired positions section, multicall polling.
- **Audience:** Takers, makers.

settlement-oracle.md

- **Purpose:** Document the SettlementPublisher oracle.
- **Covers:** Commit-reveal mechanism (commit hash, 5-minute reveal delay, 24-hour reveal window), authorized publishers, settlement price format (1e18 USD), ISettlementOracle interface, how the OptionsEngine consumes settlement prices.
- **Audience:** Developers, makers.

Makers

maker-overview.md

- **Purpose:** Comprehensive introduction to the maker role.
- **Covers:** What makers do (provide liquidity via competitive quoting), differences from AMM LP (no passive exposure, selective participation, no impermanent loss), revenue model (spread capture on fills), reliability scoring, maker-specific features (dashboard, relay, SDK).
- **Audience:** Makers.

relay-connection.md

- **Purpose:** Explain how makers connect to the WebSocket relay.
- **Covers:** Relay URL configuration, WebSocket connection handshake (SUBSCRIBE message with role + chainIds), connection acknowledgment (CONNECTED + SUBSCRIBED), ping/pong heartbeat (30s interval), auto-reconnect with exponential backoff (max 5 attempts), connection status indicators in UI.
- **Audience:** Makers, developers.

receiving-rfqs.md

- **Purpose:** Explain how RFQ broadcasts are received and filtered.
- **Covers:** REQUEST_BROADCAST message format, RFQ fields (requestId, chainId, taker, tokenIn, tokenOut, amountIn/amountOut, expiry, private flag), filtering by chain ID, filtering by token pair, private RFQ handling (only routed to allowedMakers).
- **Audience:** Makers.

submitting-quotes.md

- **Purpose:** Detail the quote submission process.
- **Covers:** Quote fields (maker, requestId, amountOut, expiry, nonce), building quote from RFQ parameters, fee consideration (feePips deduction), quote expiry (must not exceed RFQ TTL), RFQ_QUOTE message format, relay validation and QUOTE_BROADCAST.
- **Audience:** Makers.

eip712-signing.md

- **Purpose:** Technical reference for EIP-712 quote signing.
- **Covers:** EIP-712 domain (name: "HyperQuote Options", version: "1", chainId, verifyingContract), Quote type definition (all fields and types), signTypedData usage with ethers v6, signature format (65-byte r+s+v), verification via recoverQuoteSigner, on-chain ECDSA.recover in contract.
- **Audience:** Developers.

nonce-management.md

- **Purpose:** Explain the nonce system for quote invalidation.
- **Covers:** On-chain nonce per maker address, nonce must be $\geq$ current on-chain nonce, local NonceManager (init from chain, monotonic increment), quote-specific invalidation via cancelQuote(quote), bulk invalidation via incrementNonce(), resync after on-chain nonce changes.
- **Audience:** Makers, developers.

pricing-strategies.md

- **Purpose:** Guide makers on pricing approaches.
- **Covers:** Reference pricing (Black-Scholes for options, mid-price for spot), vol surface modeling (ATM vol, skew per % OTM), spread configuration, edge over fair value, market data requirements (spot price, implied vol, risk-free rate), the SDK StubPricingEngine as a starting point.
- **Audience:** Makers.

risk-management.md

- **Purpose:** Document the risk framework available to makers.
- **Covers:** RiskState tracking (per-expiry delta buckets, per-collateral notional), risk limits (max tenor 90d, max strike deviation 50%, max notional \$1M per collateral, max delta per expiry ± 100 , min premium), how recordQuote updates risk state, when to reject RFQs based on risk.
- **Audience:** Makers.

maker-sdk-quickstart.md

- **Purpose:** Get a maker bot running in 10 minutes.
- **Covers:** npm install, environment variables (MAKER_PRIVATE_KEY, CHAIN_ID, ENGINE_ADDRESS, RELAY_WS_URL, token addresses), running the relay bot, observing RFQ broadcasts and signed quote responses, verifying quotes.
- **Audience:** Developers.

maker-sdk-reference.md

- **Purpose:** Full API reference for the maker SDK.
- **Covers:** All exported modules (types, eip712, signQuote, verifyQuote, nonce, pricing, risk, rfqHash), function signatures, parameter types, return types, usage examples for each module.
- **Audience:** Developers.

auto-quoting-bot.md

- **Purpose:** Document the automated maker relay bot.
- **Covers:** makerRelay.ts pipeline (filter $\rightarrow$ price $\rightarrow$ risk check $\rightarrow$ build quote $\rightarrow$ sign $\rightarrow$ submit), configuration via environment variables, WebSocket lifecycle, keepalive ping, pipeline customization points.
- **Audience:** Developers.

private-rfq-routing.md

- **Purpose:** Explain private/selective RFQ routing for makers.

- **Covers:** How private RFQs work (taker specifies allowedMakers list), relay filters broadcasts to named makers only, import mechanism for private RFQ JSON, benefits (no public signal, exclusive quoting opportunity).
- **Audience:** Makers.

maker-reliability-score.md

- **Purpose:** Explain the maker reliability metric.
 - **Covers:** reliabilityFactor formula ($1.1 - \text{cancelRate} * 1.5$, clamped 0.5 to 1.1), how cancellations affect score, impact on league ranking, how to maintain high reliability.
 - **Audience:** Makers.
-

Venue Comparison

how-venue-comparison-works.md

- **Purpose:** Technical deep-dive into the venue comparison engine.
- **Covers:** Three venues (HyperCore Spot, HyperEVM DEX, RFQ), parallel estimation via Promise.allSettled, AbortSignal cancellation, retry logic (one retry on transient errors, 200-500ms jitter), structured result types (VenueSuccess, VenuePartial, VenueFailure).
- **Audience:** Developers.

hypercore-spot.md

- **Purpose:** Document HyperCore order book integration.
- **Covers:** Hyperliquid spot orderbook API, order book walk simulation (aggregate bid/ask levels), partial fill detection via binary search, slippage vs mid-price calculation, failure reasons (no_hl_market, insufficient depth).
- **Audience:** Developers.

hyperevm-dex.md

- **Purpose:** Document HyperEVM DEX aggregator integration.
- **Covers:** HT.xyz aggregator API proxy via /api/v1/bench/ht/price, direct route quoting, multi-hop routing (intermediates: USDC, wHYPE, USD $\blacksquare$ 0), binary search for partial fills on \$25k+ trades, route label display, failure reasons (no_dex_route).
- **Audience:** Developers.

multi-hop-routing.md

- **Purpose:** Explain multi-hop DEX route discovery.
- **Covers:** When multi-hop is attempted (direct route returns null), intermediate token priority list (USDC, wHYPE, USD $\blacksquare$ 0), two-leg chaining (tokenIn -> intermediate -> tokenOut), parallel attempts via Promise.allSettled, best result selection (highest amountOut), route label format ("TOKEN_A -> INTERMEDIATE -> TOKEN_B").

- **Audience:** Developers.

partial-fills.md

- **Purpose:** Explain partial fill detection and reporting.
- **Covers:** When partial fills occur (order book too thin, DEX liquidity insufficient), binary search algorithm for max fillable size, VenuePartial result type (filledPct, filledIn, filledOut, remainingIn, avgPrice), UI display of partial fill percentage.
- **Audience:** Takers, developers.

benchmark-methodology.md

- **Purpose:** Document how benchmark/reference prices are computed.
 - **Covers:** Mid-price benchmark source, exact-in vs exact-out benchmark computation, how RFQ price improvement is calculated against benchmarks, baseline API endpoints (/api/v1/rfq/baseline), performance tracking (/api/v1/rfq/performance).
 - **Audience:** All users.
-

Smart Contracts

contract-architecture.md

- **Purpose:** High-level overview of the smart contract system.
- **Covers:** Contract dependency graph (OptionsEngine -> ISettlementOracle, QuoteLib, CollateralMath), EIP-712 domain configuration, Ownable admin pattern, ERC-721 position minting, contract compilation (Solidity 0.8.24, via-IR optimization), OpenZeppelin dependencies.
- **Audience:** Developers.

spot-rfq-contract.md

- **Purpose:** Document the spot RFQ settlement contract.
- **Covers:** Contract ABI (from config/contracts.ts), key functions (execute, feePips), ERC-20 approval requirements, atomic swap settlement, fee deduction mechanics.
- **Audience:** Developers.

options-engine.md

- **Purpose:** Full reference for the OptionsEngine contract.
- **Covers:** execute() (quote verification, nonce check, EIP-712 signature recovery, position NFT minting, premium transfer, collateral locking), settle() (oracle price fetch, ITM verification, physical delivery, keeper fee), expirePosition() (OTM verification after settlement window, collateral return), cancelQuote(), incrementNonce(), admin functions (setAllowedCollateral, setAllowedUnderlying, setOracle, setKeeperBps, setMaxKeeperFee), events, error codes.
- **Audience:** Developers.

settlement-publisher.md

- **Purpose:** Full reference for the SettlementPublisher oracle contract.
- **Covers:** commitPrice() (phase 1: hash commit), revealPrice() (phase 2: reveal with 5-min delay, 24-hr window), getSettlementPrice() (returns price in 1e18 USD + settled boolean), authorized publisher management (addPublisher, removePublisher), ISettlementOracle interface.
- **Audience:** Developers.

quote-library.md

- **Purpose:** Document the QuoteLib library.
- **Covers:** Quote struct definition (maker, taker, underlying, collateral, isCall, isMakerSeller, strike, quantity, premium, expiry, deadline, nonce), EIP-712 typehash, hash() function, struct encoding.
- **Audience:** Developers.

collateral-math.md

- **Purpose:** Document the CollateralMath library.
- **Covers:** putCollateralRequired() formula, callSettlementCost() formula, notional() calculation, ceilDiv() utility, decimal scaling between underlying (18 dec) and collateral (6 dec), worked examples.
- **Audience:** Developers.

contract-addresses.md

- **Purpose:** List all deployed contract addresses.
- **Covers:** Mainnet HyperEVM addresses (OptionsEngine, SettlementPublisher, spot RFQ), token addresses (wHYPE, USDC, USDH, USD $\blacksquare$ 0), local/Anvil development addresses, chain ID configuration.
- **Audience:** All users.

security-model.md

- **Purpose:** Document smart contract security assumptions and design.
- **Covers:** Ownable admin pattern (no proxy, no upgradeability), EIP-712 replay protection (nonce + deadline + quote hash), commit-reveal oracle (front-running mitigation), settlement window timing, keeper incentive alignment, allowed token allowlists, audit status.
- **Audience:** Developers.

API

api-overview.md

- **Purpose:** Introduction to HyperQuote APIs.
- **Covers:** Three API surfaces (REST API on port 3000, WebSocket relay on port 8080, Terminal API on port 4200), base URLs, versioning (v1 prefix), content type (JSON), response format conventions.

- **Audience:** Developers.

authentication.md

- **Purpose:** Document API authentication methods.
- **Covers:** Bearer token auth (Authorization: Bearer hq_live_...), EIP-191 wallet signature auth, EIP-712 typed data signatures for quotes, agent registration for programmatic access, API key format and rotation.
- **Audience:** Developers.

agent-registration.md

- **Purpose:** Explain the agent API key system.
- **Covers:** POST /api/v1/agent/register (name, ownerWallet, agentWallet, roles, signature, timestamp), EIP-191 signature verification, rate limits (5/hour, 15/day per IP), API key shown once, roles (taker, maker, monitor), key rotation via /api/v1/agent/keys/rotate.
- **Audience:** Developers.

rfq-endpoints.md

- **Purpose:** Document all RFQ-related endpoints.
- **Covers:** POST /api/v1/agent/rfqs (create RFQ: tokenIn, tokenOut, amountIn/amountOut, kind, ttlSeconds, visibility, allowedMakers), GET /api/v1/rfqs (list with cursor pagination, status filter), GET /api/v1/rfqs/[id] (detail), POST /api/v1/rfqs/[id]/cancel, request/response schemas, error codes.
- **Audience:** Developers.

quote-endpoints.md

- **Purpose:** Document all quote-related endpoints.
- **Covers:** POST /api/v1/quotes (submit signed quote: rfqId, quote JSON, token), GET /api/v1/agent/rfqs/[id]/quotes (list quotes for RFQ), GET /api/v1/agent/quotes (list agent's quotes), signature validation, quote expiry enforcement.
- **Audience:** Developers.

fill-endpoints.md

- **Purpose:** Document fill/execution endpoints.
- **Covers:** POST /api/v1/rfqs/[id]/fill (execute fill), GET /api/v1/fills (historical fills), POST /api/v1/agent/rfqs/[id]/fill (agent fill), on-chain confirmation tracking.
- **Audience:** Developers.

feed-and-stream.md

- **Purpose:** Document real-time feed endpoints.
- **Covers:** GET /api/v1/feed/stream (SSE stream for activity updates), GET /api/v1/agent/feed/stream (agent-specific stream), event types, reconnection handling, SSE protocol.
- **Audience:** Developers.

leaderboard-endpoints.md

- **Purpose:** Document leaderboard API.
- **Covers:** GET /api/v1/leaderboard (tab: makers/takers, window: 7d/30d/all, cursor pagination), GET /api/v1/leaderboard/me (personal rank), response schema (rank, points, notional, fills, improvement, badges).
- **Audience:** Developers.

league-endpoints.md

- **Purpose:** Document league API.
- **Covers:** GET /api/v1/league (league info), GET /api/v1/league/activity (activity feed for an address), scoring methodology reference.
- **Audience:** Developers.

badge-endpoints.md

- **Purpose:** Document NFT badge lookup API.
- **Covers:** GET /api/v1/badges/[address] (returns badge ownership: hasHypio, hasHypurr, boost multiplier), badge contract addresses (Hypio: 0x63eb..., Hypurr: 0x9125...), boost calculation.
- **Audience:** Developers.

profile-endpoints.md

- **Purpose:** Document user profile API.
- **Covers:** GET /api/v1/profile/[address] (user stats, points rank, badges, league rank, historical activity).
- **Audience:** Developers.

benchmark-endpoints.md

- **Purpose:** Document venue benchmark APIs.
- **Covers:** GET /api/v1/bench/ht/price (HT.xyz DEX quote proxy), GET /api/v1/bench/ht/purr (PURR benchmark), GET /api/v1/bench/hypercore/purr (HyperCore reference), GET /api/v1/bench/hyperbloom/price (HyperBloom baseline), GET /api/v1/rfq/baseline, GET /api/v1/rfq/performance, GET /api/v1/rfq/performance-summary.
- **Audience:** Developers.

sor-endpoints.md

- **Purpose:** Document Smart Order Router API.
- **Covers:** GET /api/v1/sor/quote (aggregated quote), GET /api/v1/sor/pools (pool listing), GET /api/v1/sor/pools/scan (pool discovery), GET /api/v1/sor/pool-state (current state), POST /api/v1/sor/pool-state/refresh (cache refresh), GET /api/v1/sor/protocols (protocol listing), GET /api/v1/sor/health, GET /api/v1/sor/coverage.
- **Audience:** Developers.

relay-websocket-protocol.md

- **Purpose:** Full reference for the WebSocket relay protocol.
- **Covers:** Connection URL (ws://relay:8080), handshake (SUBSCRIBE with role + chainIds -> CONNECTED + SUBSCRIBED), message types (RFQ_SUBMIT, RFQ_BROADCAST, QUOTE_SUBMIT, QUOTE_BROADCAST, PING, PONG, ERROR), message schemas with field-level documentation, RFQ signature (EIP-191 of rfqId bytes), quote signature (EIP-712 typed data), validation rails (V1 allowlists, expiry constraints, rate limits), REST endpoints (/rfqs, /quotes, /health), TTL and cleanup (60s default).
- **Audience:** Developers.

terminal-api.md

- **Purpose:** Document the Terminal market data API.
- **Covers:** Base URL (port 4200), authentication (optional Bearer token), GET /options/tape (unified trade tape, filters: underlying, limit, offset, liquidityGuess, venue), GET /options/ladder (strike ladder by underlying + expiry), GET /options/venues (venue snapshot), GET /options/strike-detail (single strike pricing), field-level schemas.
- **Audience:** Developers.

rate-limits.md

- **Purpose:** Document all rate limiting rules.
- **Covers:** Relay: 30 messages/minute per IP, Agent registration: 5/hour + 15/day per IP, RFQ limits: 10 public + 20 private per wallet, API key rate limits per agent tier.
- **Audience:** Developers.

error-codes.md

- **Purpose:** Enumerate all API error codes and their meanings.
 - **Covers:** HTTP status codes, relay ERROR message codes, contract revert reasons, validation error schemas, troubleshooting guidance.
 - **Audience:** Developers.
-

Fees

fee-structure.md

- **Purpose:** Overview of all fees in the protocol.
- **Covers:** Spot RFQ fees (feePips from contract, default 250 bps), options keeper fees (keeperBps, default 10 bps = 0.10%, max 50 bps), no hidden fees, fee visibility in quote display, how fees are deducted.
- **Audience:** All users.

maker-fees.md

- **Purpose:** Detail fees from the maker perspective.

- **Covers:** How makers should account for feePips in their quotes, fee deduction from amountOut, net execution amount.
- **Audience:** Makers.

taker-fees.md

- **Purpose:** Detail fees from the taker perspective.
- **Covers:** Fee display in quote comparison, effective rate after fees, comparison against AMM swap fees.
- **Audience:** Takers.

keeper-fees-options.md

- **Purpose:** Document the options keeper fee mechanism.
 - **Covers:** Keeper fee formula ($\min(\text{notional} * \text{keeperBps} / 10000, \text{maxKeeperFee})$), who pays (deducted from seller proceeds at settlement), keeper incentive (anyone can call settle() and earn the fee), admin configurability (setKeeperBps, setMaxKeeperFee).
 - **Audience:** All users.
-

Points

points-overview.md

- **Purpose:** Introduce the points program.
- **Covers:** Purpose (reward meaningful liquidity activity), how points are earned (on each filled trade), leaderboard ranking, point decay (sublinear scaling discourages wash trading), NFT boosts.
- **Audience:** All users.

base-points-formula.md

- **Purpose:** Document the base points calculation.
- **Covers:** Formula: $\text{base} = (\text{notionalUsd} / 1000) ^ {0.9}$, worked examples (\$1k -> 1.0 pts, \$10k -> 7.94 pts, \$100k -> 63.1 pts, \$1M -> 501 pts), why sublinear (diminishing returns on size to prevent whale dominance).
- **Audience:** All users.

multipliers.md

- **Purpose:** Document all points multipliers.
- **Covers:** Improvement multiplier ($1 + \text{clamp}(\text{improvementBps}, -20, +50) / 100$, range 0.8 to 1.6, missing benchmark = 0.9 penalty), privacy multiplier (1.10x for private fills $\geq \$50k$), pair-repeat decay (<10 : 1.0, $10-19$: 0.5, ≥ 20 : 0.25), product clamp range (0.5 to 3.0).
- **Audience:** All users.

nft-boosts.md

-
- **Purpose:** Document NFT-based points boosts.
 - **Covers:** Eligible collections (Lucky Hypio Winners: 1.25x, Hypurr: 1.5x, both: 2.0x), contract addresses, how boost is applied (multiplied after base * multipliers), how to check eligibility.
 - **Audience:** All users.

anti-gaming-guards.md

- **Purpose:** Explain anti-wash-trading protections.
- **Covers:** Pair-repeat decay (counts trades per taker-maker pair, progressive penalty after 10 repeats), sublinear base formula (diminishing returns on size), missing benchmark penalty (0.9x when no venue comparison available).
- **Audience:** All users.

points-faq.md

- **Purpose:** Common questions about the points system.
 - **Covers:** When are points awarded, can points be lost, how does the leaderboard update, what counts as a fill, do cancelled RFQs affect points, how are badges verified.
 - **Audience:** All users.
-

League

league-overview.md

- **Purpose:** Introduce the Liquidity League competitive ranking.
- **Covers:** What the league is (competitive ranking of makers and takers), how it differs from points (score-based vs points-based, incorporates reliability and improvement), ranking periods (7d, 30d, all-time), minimum size filters.
- **Audience:** All users.

maker-scoring.md

- **Purpose:** Document maker league score formula.
- **Covers:** Formula: $\text{score} = \text{filledNotional} (1 + \text{avgImprovementBps}/100) \text{reliabilityFactor} \text{privacyFactor}$, $\text{reliabilityFactor} = \text{clamp}(1.1 - \text{cancelRate}, 1.5, 0.5, 1.1)$, $\text{privacyFactor} = 1 + \min(\text{privateShare}, 0.5) * 0.1$, worked examples.
- **Audience:** Makers.

taker-scoring.md

- **Purpose:** Document taker league score formula.
- **Covers:** Formula: $\text{score} = \text{filledNotional} (1 + \text{avgImprovementBps}/120) \text{privacyFactor}$, privacyFactor same as maker, worked examples, how taker scoring differs from maker scoring.
- **Audience:** Takers.

ranking-periods.md

- **Purpose:** Explain league time windows.
- **Covers:** 7-day rolling, 30-day rolling, all-time, how fills are aggregated per window, when rankings refresh, pair-repeat decay application per window.
- **Audience:** All users.

league-activity-feed.md

- **Purpose:** Document the league activity detail view.
 - **Covers:** GET /api/v1/league/activity, recent fills per address (counterparty, timestamp, notional, improvement, isPrivate), detail sheet in league UI, link to full profile page.
 - **Audience:** All users.
-

Risk

risk-overview.md

- **Purpose:** High-level summary of all protocol risks.
- **Covers:** Categorized risk list (smart contract, liquidity, execution, oracle, infrastructure, market), risk mitigation strategies, user responsibilities, protocol limitations.
- **Audience:** All users.

smart-contract-risk.md

- **Purpose:** Document smart contract specific risks.
- **Covers:** Immutable deployment (no proxy/upgrade, reduces governance risk but prevents bug fixes), Solidity 0.8.24 (overflow protection built-in), OpenZeppelin dependency, Ownable admin key risk, audit status, EIP-712 signature security assumptions.
- **Audience:** All users.

liquidity-risk.md

- **Purpose:** Document liquidity availability risks.
- **Covers:** RFQ quotes are not guaranteed (makers may not respond), quote quality depends on active maker competition, thin markets may produce no quotes or poor pricing, venue comparison may show superior AMM pricing for small trades.
- **Audience:** Takers.

quote-expiry-risk.md

- **Purpose:** Document quote expiration risks.
- **Covers:** Quotes have TTL deadlines, price movement between quote receipt and execution, expired quotes cannot be filled, network congestion may prevent timely execution, urgency indicators in UI.
- **Audience:** Takers.

oracle-risk.md

- **Purpose:** Document settlement oracle risks (options).
- **Covers:** SettlementPublisher is a trusted oracle (authorized publishers only), commit-reveal delay mitigates front-running but doesn't eliminate publisher trust, 24-hour settlement window, what happens if no settlement price is published (positions cannot be settled, must wait for expirePosition after window).
- **Audience:** All users.

relay-dependency.md

- **Purpose:** Document relay infrastructure risks.
- **Covers:** Relay is off-chain infrastructure (WebSocket server), relay downtime prevents new RFQ/quote matching, existing on-chain positions are unaffected by relay outage, reconnection logic (exponential backoff, max 5 attempts), relay does not hold funds.
- **Audience:** All users.

volatility-risk.md

- **Purpose:** Document market volatility risks.
- **Covers:** Price movement between RFQ submission and fill execution, options settlement price may differ from expected value, implied volatility changes affect option pricing, makers may withdraw quotes during volatility.
- **Audience:** All users.

routing-limitations.md

- **Purpose:** Document venue comparison and routing limitations.
- **Covers:** DEX route discovery depends on HT.xyz aggregator availability, multi-hop routing limited to three intermediates (USDC, wHYPE, USD $\blacksquare$ 0), HyperCore book depth may be insufficient for large sizes, partial fill detection uses binary search (approximate, not exact), transient API failures may cause missing venue estimates.
- **Audience:** Takers.

counterparty-risk.md

- **Purpose:** Document counterparty considerations.
- **Covers:** Atomic settlement eliminates traditional counterparty risk for spot swaps, options have time-delayed settlement (counterparty trust mitigated by locked collateral), maker quote signatures are binding once submitted on-chain, nonce invalidation allows makers to cancel unexecuted quotes.
- **Audience:** All users.

FAQ

general-faq.md

- **Purpose:** Answer common general questions.

- **Covers:** What is HyperQuote, how does it differ from a DEX, what chain does it run on, is it permissionless, what tokens are supported, is there a token, what are the fees.
- **Audience:** All users.

taker-faq.md

- **Purpose:** Answer taker-specific questions.
- **Covers:** How long do I wait for quotes, what if no maker responds, can I cancel an RFQ, how do I know if a quote is good, what is price improvement, what happens if my transaction fails.
- **Audience:** Takers.

maker-faq.md

- **Purpose:** Answer maker-specific questions.
- **Covers:** How do I start making, do I need to run a server, what is the minimum capital, how are quotes signed, can I quote selectively, what happens if I cancel too much.
- **Audience:** Makers.

options-faq.md

- **Purpose:** Answer options-specific questions.
- **Covers:** What expiries are available, how is settlement price determined, what if I'm ITM at expiry, who calls settle, what are keeper fees, can I transfer my position NFT.
- **Audience:** All users.

technical-faq.md

- **Purpose:** Answer technical/developer questions.
- **Covers:** How do I run locally, what is the relay, how does EIP-712 signing work, how do I register an agent, what are the rate limits, how do I get historical data.
- **Audience:** Developers.

Links

official-links.md

- **Purpose:** Centralized list of all official resources.
- **Covers:** App URL, documentation URL, GitHub, smart contract addresses, block explorer links, relay endpoints, support channels.
- **Audience:** All users.

contract-addresses.md

- **Purpose:** Reference page for all deployed addresses.
- **Covers:** Mainnet and testnet addresses for all contracts and tokens, chain IDs, RPC URLs.

- **Audience:** Developers.

brand-assets.md

- **Purpose:** Official brand resources.
 - **Covers:** Logo files, color palette, usage guidelines.
 - **Audience:** Partners, integrators.
-
-

3. Required Diagrams

Protocol Flow Diagrams

1. RFQ Lifecycle (Spot)

- Taker creates RFQ -> Relay broadcasts -> Makers quote -> Taker compares -> On-chain fill -> Settlement
- Show: UI, relay, smart contract, and venue comparison engine as swim lanes

2. RFQ Lifecycle (Options)

- Taker submits options RFQ -> Relay broadcasts -> Maker signs EIP-712 quote -> Taker executes -> Position NFT minted -> (time passes) -> Settlement or Expiry
- Show: premium transfer, collateral locking, oracle publish, physical delivery

3. Maker Quoting Flow

- Relay connection -> RFQ received -> Filter -> Price (Black-Scholes / spread) -> Risk check -> Build quote -> EIP-712 sign -> Submit -> Relay broadcasts
- Show: decision points where RFQ may be rejected

4. Venue Comparison Engine

- RFQ parameters -> Parallel fetch (HyperCore book walk, HT.xyz DEX quote, multi-hop attempt, binary search partial) -> Structured results (success/partial/failure) -> UI comparison panel
- Show: retry logic, abort signal, result union types

5. On-Chain Settlement Flow (Spot)

- Taker approve(tokenIn) -> execute(quote, signature) -> Contract verifies sig + nonce -> Atomic swap (tokenIn taker->maker, tokenOut maker->taker) -> Fee deduction -> Event emission

6. On-Chain Settlement Flow (Options)

- execute(quote, sig) -> Verify (nonce, sig, expiry, allowlists) -> Mint ERC-721 to buyer -> Premium: buyer->seller -> Lock collateral: seller->contract
- Then: settle(positionId) -> Oracle price -> ITM check -> Physical delivery -> Keeper fee -> Burn NFT
- Or: expirePosition(positionId) -> After 24h window -> OTM check -> Return collateral -> Burn NFT

7. Public vs Private RFQ Routing

- Public: Taker -> Relay -> All connected makers -> Multiple quotes -> Taker selects best
- Private: Taker (with allowedMakers) -> Relay -> Only named makers -> Exclusive quotes -> Taker selects

- Show: relay filtering logic

8. Settlement Oracle (Commit-Reveal)

- Publisher -> commitPrice(hash) -> 5 min delay -> revealPrice(asset, expiry, price, salt) -> Price stored
- > OptionsEngine.settle() reads price
- Show: timing constraints (5 min reveal delay, 24 hr reveal window)

Architecture Diagrams

9. System Architecture

- Components: Next.js UI, WebSocket Relay, Spot RFQ Contract, OptionsEngine Contract, SettlementPublisher, HyperCore API, HT.xyz Aggregator, Terminal API, Terminal Ingest, Prisma DB
- Show: data flow arrows, on-chain vs off-chain boundary

10. Data Flow: Terminal Ingest Pipeline

- Sources: Derive API (tickers + trades), HyperEVM events (executions + settlements), Hyperliquid (spot prices)
- Pipeline: Pollers -> Normalized tables (unified_tape, hq_executions, hl_spot) -> Terminal API -> UI Terminal
- Show: poll intervals, retention policies

Scoring Diagrams

11. Points Calculation Flow

- Fill event -> Compute notionalUSD -> Base points (sublinear) -> Apply multipliers (improvement, privacy, pair-repeat decay, clamp) -> Apply NFT boost -> Final points
- Show: formula at each step with example values

12. League Scoring Flow

- Aggregate fills per address -> Group by pair -> Apply pair-repeat decay -> Sum decayed notionals -> Compute weighted avg improvement -> Apply reliability + privacy factors -> Final league score
- Show: maker vs taker formula differences

Token & State Diagrams

13. EIP-712 Quote Signing

- Domain (name, version, chainId, verifyingContract) + Quote struct fields -> EIP-712 hash -> Wallet signTypedData -> 65-byte signature (r, s, v) -> On-chain ECDSA.recover -> Verify signer == maker

14. Options Position State Machine

- States: Active (post-execute) -> Settled (ITM, within 24h window) or Expired (OTM, after 24h window)
- Transitions with guard conditions (settlement price vs strike, time constraints)

4. API Documentation Plan

Section 1: REST API (Port 3000)

1.1 Authentication

- Bearer token auth (hq_live_... prefix)
- EIP-191 signature auth for registration
- Role-based access (taker, maker, monitor)

1.2 Agent Management

- POST /api/v1/agent/register
- GET /api/v1/agent/auth
- POST /api/v1/agent/keys/rotate
- GET /api/v1/agent/tokens
- GET /api/v1/agent/venues
- GET /api/v1/agent/contract

1.3 RFQ Operations

- POST /api/v1/agent/rfqs (create)
- GET /api/v1/agent/rfqs (list with source/status/limit filters)
- GET /api/v1/agent/rfqs/[id] (detail)
- POST /api/v1/rfqs/[id]/cancel
- GET /api/v1/rfqs (public list with cursor pagination)

1.4 Quote Operations

- POST /api/v1/quotes (submit signed quote)
- GET /api/v1/agent/rfqs/[id]/quotes (list quotes for RFQ)
- GET /api/v1/agent/quotes (list agent's quotes)

1.5 Fill Operations

- POST /api/v1/rfqs/[id]/fill
- POST /api/v1/agent/rfqs/[id]/fill
- GET /api/v1/fills (historical fills)

1.6 Real-Time Feeds

- GET /api/v1/feed/stream (SSE)
- GET /api/v1/agent/feed/stream (SSE)
- GET /api/rfq/stream (legacy SSE)

1.7 Leaderboard & League

-
- GET /api/v1/leaderboard (tab, window, cursor)
 - GET /api/v1/leaderboard/me
 - GET /api/v1/league
 - GET /api/v1/league/activity
 - GET /api/v1/agent/leaderboard

1.8 Profiles & Badges

- GET /api/v1/profile/[address]
- GET /api/v1/badges/[address]

1.9 Benchmarks & Performance

- GET /api/v1/bench/ht/price
- GET /api/v1/bench/ht/purr
- GET /api/v1/bench/hypercore/purr
- GET /api/v1/bench/hyperbloom/price
- GET /api/v1/rfq/baseline
- GET /api/v1/rfq/performance
- GET /api/v1/rfq/performance-summary

1.10 Smart Order Router

- GET /api/v1/sor/quote
- GET /api/v1/sor/pools
- GET /api/v1/sor/pools/scan
- GET /api/v1/sor/pool-state
- POST /api/v1/sor/pool-state/refresh
- GET /api/v1/sor/protocols
- POST /api/v1/sor/protocols/sync
- GET /api/v1/sor/health
- GET /api/v1/sor/coverage

1.11 System

- GET /api/health
- GET /api/explorer/contract-status
- GET /api/hyperliquid/orderbook

Section 2: WebSocket Relay (Port 8080)

2.1 Connection

- URL, handshake, subscribe message, acknowledgments

2.2 Message Types

- RFQ_SUBMIT, RFQ_BROADCAST, QUOTE_SUBMIT, QUOTE_BROADCAST, PING, PONG, ERROR
- Full schema for each message type

2.3 Validation

- Signature verification (EIP-191 for RFQs, EIP-712 for quotes)
- V1 allowlists (underlying, collateral)
- Expiry constraints
- Rate limits

2.4 REST Endpoints

- GET /rfqs (active RFQs)
- GET /quotes?rfqId=... (quotes for RFQ)
- GET /health (relay status)

Section 3: Terminal API (Port 4200)

3.1 Trade Tape

- GET /options/tape (filters, pagination, response schema)

3.2 Strike Ladder

- GET /options/ladder (required params, per-strike fields)

3.3 Venue Snapshot

- GET /options/venues (per-expiry data)

3.4 Strike Detail

- GET /options/strike-detail (single instrument pricing)
-
-

5. Smart Contract Documentation Plan

5.1 Contract Architecture

- Dependency graph: OptionsEngine -> {ISettlementOracle, QuoteLib, CollateralMath, ERC721, Ownable, ECDSA, EIP712}
- Spot RFQ contract (separate from options)

-
- Compilation: Solidity 0.8.24, via-IR, OpenZeppelin v4/v5

5.2 OptionsEngine (Core)

- Constructor parameters and initialization
- State variables: positions mapping, nonces mapping, usedQuotes mapping, allowedCollateral, allowedUnderlying, oracle address, keeperBps, maxKeeperFee
- Functions: execute, settle, expirePosition, cancelQuote, incrementNonce, admin setters
- Events: QuoteExecuted, PositionSettled, PositionExpired, KeeperFeePaid, QuoteCancelled, NonceIncremented
- Errors: InvalidSignature, QuoteAlreadyUsed, NonceTooLow, InvalidExpiry, UnsupportedCollateral, UnsupportedUnderlying, PositionNotActive, NotITM, NotOTM, SettlementWindowActive, SettlementWindowExpired

5.3 SettlementPublisher (Oracle)

- State: commits mapping, prices mapping, authorized publishers
- Functions: commitPrice, revealPrice, getSettlementPrice, hasPriceFor, addPublisher, removePublisher
- Constants: REVEAL_DELAY (5 min), REVEAL_WINDOW (24 hr)
- Events: SettlementPricePublished, PriceCommitted

5.4 Libraries

- QuoteLib: Quote struct, EIP-712 typehash, hash()
- CollateralMath: putCollateralRequired, callSettlementCost, notional, ceilDiv

5.5 Interfaces

- ISettlementOracle: getSettlementPrice, hasPriceFor
- IPositions: settle, expirePosition, getPosition
- IQuoteVerifier: execute, cancelQuote, incrementNonce, nonces, isQuoteUsed, hashQuote

5.6 Execution Flow (Documented Step-by-Step)

1. Taker calls execute(quote, signature)
2. Contract hashes quote struct (EIP-712)
3. Contract verifies quote not already used
4. Contract verifies nonce $\geq$ maker's current nonce
5. Contract recovers signer from signature, asserts $==$ quote.maker
6. Contract validates expiry ($\geq 24h$, $\leq 90d$, 08:00 UTC, deadline not passed)
7. Contract validates collateral and underlying in allowlists
8. Contract validates taker (quote.taker $==$ address(0) or $==$ msg.sender)
9. Contract marks quote as used

-
10. Contract mints ERC-721 position to buyer (quote.maker in V1)
 11. Contract transfers premium from buyer to seller
 12. Contract locks collateral from seller into contract
 13. Contract emits QuoteExecuted event

5.7 Contract Addresses

- Mainnet HyperEVM deployments
- Testnet/local Anvil addresses
- Token addresses (wHYPE, USDC, USDH, USD■0)

5.8 Upgradeability Model

- No proxy pattern detected
- Ownable for admin functions only
- Token allowlist management (add/remove collateral and underlying)
- Oracle address swappable (setOracle)
- Fee parameters adjustable (setKeeperBps, setMaxKeeperFee)
- No contract upgrade path (intentional immutability for trust)

5.9 Security Assumptions

- EIP-712 signatures cannot be forged
 - Nonce system prevents replay
 - Commit-reveal oracle prevents settlement price front-running
 - ERC-721 positions are transferable (holder can settle)
 - Ownable admin can change allowlists and fees but cannot access locked collateral
 - 24-hour settlement window is sufficient for ITM settlement
 - Keeper incentive aligns settlement execution with third parties
-
-

6. Risk Documentation Plan

Smart Contract Risk

- **Immutable contracts:** No upgrade path means bugs cannot be patched post-deployment
- **Admin key risk:** Owner can modify allowlists, oracle address, and fee parameters
- **ERC-721 transfer risk:** Position NFTs can be transferred; new holder inherits settlement rights
- **Reentrancy:** Mitigated by checks-effects-interactions pattern and ERC-20 standard compliance
- **Integer overflow:** Solidity 0.8.24 has built-in overflow protection

-
- **Audit status:** Document current audit state and any known findings

Liquidity Risk

- **No guaranteed quotes:** Makers are not obligated to respond to RFQs
- **Market conditions:** Volatile markets may reduce maker participation
- **Thin pairs:** Low-liquidity token pairs may receive no quotes
- **Venue comparison:** AMMs may offer better pricing for small trades where RFQ overhead exceeds spread improvement
- **Maker concentration:** Single-maker dependency if few makers are active

Quote Expiry Risk

- **Time decay:** Quotes expire after their deadline timestamp
- **Network latency:** Blockchain confirmation time may exceed quote validity
- **Gas price spikes:** High gas costs may make execution uneconomical
- **Stale pricing:** Market may move between quote generation and on-chain execution
- **Front-running:** Public mempool transactions may be front-run (mitigated by signed quotes targeting specific taker)

Oracle Risk (Options)

- **Trusted publisher:** Settlement prices come from authorized publishers only
- **Publish failure:** If no price is published, ITM positions cannot be settled within the window
- **Price manipulation:** Commit-reveal mitigates but does not eliminate oracle manipulation
- **Timing risk:** 5-minute reveal delay creates a window where price is committed but not yet public
- **Single oracle:** No multi-oracle redundancy in V1

Relay Dependency Risk

- **Single point of failure:** Relay downtime prevents new RFQ matching
- **Censorship risk:** Relay operator could theoretically filter RFQs or quotes
- **Data loss:** In-memory storage means relay restart loses active RFQs
- **No fund risk:** Relay never holds or controls user funds
- **Mitigation:** On-chain positions and settlements function independently of relay

Volatility Risk

- **Price movement:** Rapid price changes between RFQ and execution
- **Options exposure:** Locked collateral may become insufficient during extreme moves (mitigated by physical settlement model)
- **IV changes:** Implied volatility shifts affect options fair value between quote and execution

-
- **Maker withdrawal:** Makers may pull quotes during high volatility using nonce invalidation

Routing Limitations

- **DEX aggregator dependency:** HT.xyz availability affects venue comparison accuracy
- **Multi-hop coverage:** Limited to three intermediate tokens (USDC, wHYPE, USD■0)
- **Partial fill approximation:** Binary search finds approximate, not exact, max fillable size
- **Cross-venue timing:** Venue estimates may be stale by the time user executes (30-second refresh)
- **Stable assumptions:** Homepage benchmark assumes 1:1 for USDH/USDC/USD■0 conversions

Counterparty Risk

- **Spot swaps:** Atomic settlement eliminates counterparty risk
- **Options:** Collateral locked on-chain eliminates default risk for the collateralized leg
- **Keeper dependency:** ITM settlement requires someone to call settle() within the 24-hour window
- **Keeper incentive alignment:** Keeper fee creates economic motivation for third-party settlement
- **Unsigned taker:** Open quotes (taker = address(0)) can be executed by anyone

API and Infrastructure Risk

- **Rate limiting:** Excessive requests may be throttled (30 msg/min relay, 5 req/hour agent)
- **API key security:** Lost or leaked API keys grant agent access until rotated
- **SSE stream reliability:** Network interruptions may cause missed events
- **Data freshness:** Terminal data has polling intervals (3-5 seconds) and may lag real-time prices
- **Retention policy:** Historical data is pruned (hl_spot: 7 days, derive_trades: 30 days)